

A *Java Enum* is a special Java type used to define collections of constants. More precisely, a Java enum type is a special kind of Java class. An enum can contain constants, methods etc.

Enum Example

Here is a simple Java enum example:

```
public enum Level {  
    HIGH,  
    MEDIUM,  
    LOW  
}
```

Notice the `enum` keyword which is used in place of `class` or `interface`. The Java `enum` keyword signals to the Java compiler that this type definition is an enum.

You can refer to the constants in the enum above like this:

```
Level level = Level.HIGH;
```

Notice how the `level` variable is of the type `Level` which is the Java enum type defined in the example above. The `level` variable can take one of the `Level` enum constants as value (`HIGH`, `MEDIUM` or `LOW`). In this case `level` is set to `HIGH`.

Enums in if Statements

Since Java enums are constants you will often have to compare a variable pointing to an enum constant against the possible constants in the enum type. Here is an example of using a Java enum in an `if`-statement:

```
Level level = ... //assign some Level constant to it  
  
if( level == Level.HIGH) {  
  
} else if( level == Level.MEDIUM) {  
  
} else if( level == Level.LOW) {  
  
}
```

```
}
```

This code compares the `level` variable against each of the possible enum constants in the `Level` enum.

If one of the enum values occur more often than the others, checking for that value in the first `if`-statement will result in better performance, as less comparison on average are executed. This is not a big difference though, unless the comparisons are executed a lot.

Enums in switch Statements

If your Java enum types contain a lot constants and you need to check a variable against the values as shown in the previous section, using a Java `switch` statement might be a good idea.

You can use enums in switch statements like this:

```
Level level = ... //assign some Level constant to it

switch (level) {
    case HIGH      : ...; break;
    case MEDIUM   : ...; break;
    case LOW       : ...; break;
}
```

Replace the `...` with the code to execute if the `level` variable matches the given Level constant value. The code could be a simple Java operation, a method call etc.

Enum Iteration

You can obtain an array of all the possible values of a Java enum type by calling its static `values()` method. All enum types get a static `values()` method automatically by the Java compiler. Here is an example of iterating all values of an enum:

```
for (Level level : Level.values()) {
    System.out.println(level);
}
```

Running this Java code would print out all the enum values. Here is the output:

```
HIGH  
MEDIUM  
LOW
```

Notice how the names of the constants themselves are printed out. This is one area where Java enums are different than `static final` constants.

Enum toString()

An enum class automatically gets a `toString()` method in the class when compiled. The `toString()` method returns a string value of the name of the given enum instance. Here is an example:

```
String levelText = Level.HIGH.toString();
```

The value of the `levelText` variable after execution of the above statement will be the text `HIGH`.

Enum Printing

If you print an enum, like this:

```
System.out.println(Level.HIGH);
```

Then the `toString()` method will get called behind the scenes, so the value that will be printed out is the textual name of the enum instance. In other words, in the example above the text `HIGH` would have been printed.

Enum valueOf()

An enum class automatically gets a static `valueOf()` method in the class when compiled. The `valueOf()` method can be used to obtain an instance of the enum class for a given String value. Here is an example:

```
Level level = Level.valueOf("HIGH");
```

The `level` variable will point to the `Level.HIGH` after executing this line.

Enum Fields

You can add fields to a Java enum. Thus, each constant enum value gets these fields. The field values must be supplied to the constructor of the enum when defining the constants. Here is an example:

```
public enum Level {  
    HIGH (3), //calls constructor with value 3  
    MEDIUM(2), //calls constructor with value 2  
    LOW (1) //calls constructor with value 1  
    ; // semicolon needed when fields / methods follow  
  
    private final int levelCode;  
  
    private Level(int levelCode) {  
        this.levelCode = levelCode;  
    }  
}
```

Notice how the Java enum in the example above has a constructor which takes an `int`. The enum constructor sets the `int` field. When the constant enum values are defined, an `int` value is passed to the enum constructor.

The enum constructor must be `private`. You cannot use `public` or `protected` constructors for a Java enum. If you do not specify an access modifier the enum constructor it will be implicitly `private`.

Enum Methods

You can add methods to a Java enum too. Here is an example:

```
public enum Level {  
    HIGH (3), //calls constructor with value 3  
    MEDIUM(2), //calls constructor with value 2  
    LOW (1) //calls constructor with value 1  
    ; // semicolon needed when fields / methods follow
```

```

    private final int levelCode;

    Level(int levelCode) {
        this.levelCode = levelCode;
    }

    public int getLevelCode() {
        return this.levelCode;
    }
}

```

You call a Java enum method via a reference to one of the constant values. Here is Java enum method call example:

```

Level level = Level.HIGH;

System.out.println(level.getLevelCode());

```

This code would print out the value 3 which is the value of the `levelCode` field for the enum constant `HIGH`.

You are not restricted to simple getter and setter methods. You can also create methods that make calculations based on the field values of the enum constant. If your fields are not declared `final` you can even modify the values of the fields (although that may not be so good an idea, considering that the enums are supposed to be constants).

Enum Abstract Methods

It is possible for a Java enum class to have abstract methods too. If an enum class has an abstract method, then each instance of the enum class must implement it. Here is a Java enum abstract method example:

```

public enum Level {
    HIGH{
        @Override
        public String asLowerCase() {
            return HIGH.toString().toLowerCase();
        }
    },
    MEDIUM{
        @Override
        public String asLowerCase() {
            return MEDIUM.toString().toLowerCase();
        }
    }
}

```

```

        }
    },
    LOW{
        @Override
        public String asLowerCase() {
            return LOW.toString().toLowerCase();
        }
    };

    public abstract String asLowerCase();
}

```

Notice the abstract method declaration at the bottom of the enum class. Notice also how each enum instance (each constant) defines its own implementation of this abstract method. Using an abstract method is useful when you need a different implementation of a method for each instance of a Java enum.

Enum Implementing Interface

A Java `Enum` can implement a **Java Interface** in case you feel that makes sense in your situation. Here is an example of a Java `Enum` implementing an interface:

```

public enum EnumImplementingInterface implements MyInterface {
    FIRST("First Value"), SECOND("Second Value");

    private String description = null;

    private EnumImplementingInterface(String desc){
        this.description = desc;
    }

    @Override
    public String getDescription() {
        return this.description;
    }
}

```

It is the method `getDescription()` that comes from the interface `MyInterface`.

Implementing an interface with an `Enum` could be used to implement a set of different `Comparator` constants which can be used to sort collections of objects. Sorting of objects in Java is explained in more detail in the **Java Collection Sorting Tutorial**.

EnumSet

Java contains a special **Java Set** implementation called `EnumSet` which can hold enums more efficiently than the standard Java Set implementations. Here is how you create an instance of an `EnumSet` :

```
EnumSet<Level> enumSet = EnumSet.of(Level.HIGH, Level.MEDIUM);
```

Once created, you can use the `EnumSet` just like any other Set.

EnumMap

Java also contains a special **Java Map** implementation which can use Java enum instances as keys. Here is a Java `EnumMap` example:

```
EnumMap<Level, String> enumMap = new EnumMap<Level, String>(Level.class);
enumMap.put(Level.HIGH, "High level");
enumMap.put(Level.MEDIUM, "Medium level");
enumMap.put(Level.LOW, "Low level");

String levelValue = enumMap.get(Level.HIGH);
```

Enum Miscellaneous Details

Java enums extend the `java.lang.Enum` class implicitly, so your enum types cannot extend another class.

If a Java enum contains fields and methods, the definition of fields and methods must always come *after* the list of constants in the enum. Additionally, the list of enum constants must be terminated by a semicolon;